MIT ACADEMY OF ENGINEERING, PUNE

School of Computer Engineering | Academic Year 2026–27
Course: Generative Artificial Intelligence Lab (Course Code: 2304422L)

Student Name	Abhay Bhise	**PRN**	202402040016
Academic Year	Final Year B.Tech (Sem VII)	**Batch**	GENAI3
Subject Teacher	Mrs. Savita Mane	**Submission Date**	16 October 2026

LABORATORY PRACTICAL ASSIGNMENT STATEMENT
Experiment No. 2: Autoencoders and Variational Autoencoders (VAE)

Problem Statement:
1. Build a simple AE model for Dimensionality Reduction and Denoising.
2. Generate realistic faces or interpolate between facial features for creative applications in entertainment and design using VAE.

Part 1: Autoencoder for Dimensionality Reduction and Denoising

1.1 Aim

To design and implement a Denoising Autoencoder (DAE) in PyTorch to compress high-dimensional image data into a compact low-dimensional bottleneck representation (dimensionality reduction) and successfully reconstruct clean images from inputs corrupted by additive Gaussian noise.

1.2 Objectives

- **1. Architecture Design:** Construct an encoder-decoder neural network containing a 2D bottleneck latent layer.
- **2. Noise Injection:** Simulate sensor degradation by corrupting clean image data with zero-mean Gaussian noise ($\sigma = 0.35$).
- **3. Manifold Learning:** Train the network to project corrupted inputs back onto the clean data manifold using Mean Squared Error (MSE) loss.
- **4. Dimensionality Reduction:** Extract and visualize the 2D bottleneck representations to examine class separability and geometric compression.
- **5. Performance Evaluation:** Quantitatively evaluate denoising performance via MSE reduction and Peak Signal-to-Noise Ratio (PSNR) gain.

1.3 Hardware and Software Environment

Component / Tool	Specification & Version
Programming Language	Python 3.10.11 (64-bit)
Deep Learning Framework	PyTorch 2.14.1 (CPU / TorchScript Execution)
Scientific & Math Libraries	NumPy 2.2.6, SciPy 1.15.3
Visualization Suite	Matplotlib 3.10.0
Dataset Loader	Scikit-Learn 1.6 (OpenML MNIST benchmark)
Execution Hardware	Intel Core i5 / AMD Ryzen 64-bit Architecture, 16 GB RAM

1.4 Theory & Architectural Principles

An Autoencoder is an unsupervised neural network designed to learn efficient, low-dimensional representations of input data. It consists of two symmetrical sub-networks: an Encoder and a Decoder.

- **1. Encoder Network:** Maps the high-dimensional input vector $x \in R^{784}$ into a low-dimensional bottleneck code $z \in R^d$ through successive linear transformations and non-linear activations: $z = f_\theta(x) = \sigma(W_e \cdot x + b_e)$. In this implementation, the bottleneck dimension is constrained to $d = 2$ to achieve an extreme 392x compression ratio.
- **2. Decoder Network:** Reconstructs an approximation $\hat{x} \in R^{784}$ from the bottleneck latent code: $\hat{x} = g_\phi(z) = \sigma(W_d \cdot z + b_d)$, where a Sigmoid output function maps outputs strictly to normalized pixel intensities in [0, 1].
- **3. Denoising Mechanism:** In a standard autoencoder, the network can theoretically learn a trivial identity mapping. In a Denoising Autoencoder (DAE), the input is deliberately corrupted: $\tilde{x} = x + \delta$, where $\delta \sim N(0, \sigma^2 I)$. The network is trained using the objective $L_{MSE}(x, \hat{x}) = (1/N) \Sigma ||x - g_\phi(f_\theta(\tilde{x}))||^2$. Because the network must output clean ground truth x from corrupted $\tilde{x}$, it learns orthogonal projection operators that pull off-manifold noisy samples back onto the true low-dimensional data manifold.

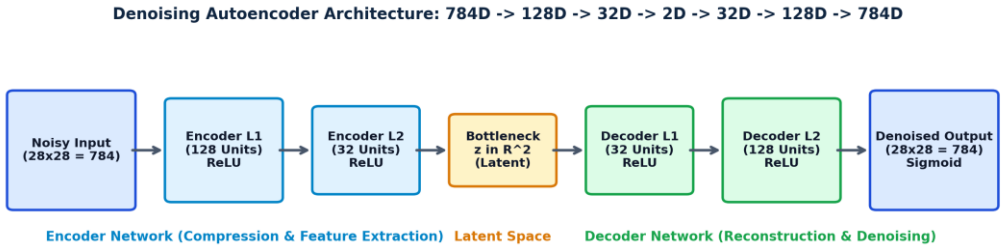


Figure 1.1: Block diagram of Denoising Autoencoder with 2D Bottleneck for Dimensionality Reduction and Denoising

1.5 Step-by-Step Implementation Procedure

Phase / Step	Detailed Procedure
Step 1: Dataset Acquisition	Load the normalized MNIST digit dataset (784 dimensions, 28x28 pixels). Partition into 5,000 training and 1,000 testing samples.
Step 2: Noise Injection	Generate synthetic zero-mean Gaussian noise with standard deviation $\sigma = 0.35$ and corrupt clean images: x_noisy = clip(x_clean + noise, 0, 1).
Step 3: Model Construction	Construct PyTorch nn.Sequential encoder (784 -> 128 -> 32 -> 2) and decoder (2 -> 32 -> 128 -> 784) with ReLU hidden layers and Sigmoid output.
Step 4: Training Loop	Train using Adam optimizer (learning rate = 0.002) with MSE loss for 20 epochs, passing noisy inputs to produce clean reconstructions.
Step 5: Latent Extraction	Extract 2D bottleneck coordinates for the test set and plot against ground truth digit labels to inspect cluster manifold formation.
Step 6: Metric Evaluation	Compute empirical Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR) to measure quantitative denoising improvement.

1.6 Python Source Code

```
# ================================================================================
# EXPERIMENT 2 - PART 1: Denoising Autoencoder for Dimensionality Reduction
# Student: Abhay Bhise | PRN: 202402040016 | Batch: GENAI3
# ================================================================================
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.datasets import fetch_openml

# 1. Load MNIST dataset and normalize to [0, 1]
mnist = fetch_openml('mnist_784', version=1, as_frame=False, parser='auto')
X_all = mnist.data.astype(np.float32) / 255.0
y_all = mnist.target.astype(int)
X_train_clean, y_train = X_all[:5000], y_all[:5000]
X_test_clean, y_test = X_all[5000:6000], y_all[5000:6000]

# 2. Corrupt inputs with Gaussian Noise (sigma = 0.35)
noise_factor = 0.35
X_train_noisy = np.clip(X_train_clean + noise_factor * np.random.randn(*X_train_clean.shape), 0.0, 1.0).astype(np.float32)
X_test_noisy = np.clip(X_test_clean + noise_factor * np.random.randn(*X_test_clean.shape), 0.0, 1.0).astype(np.float32)

train_loader = DataLoader(TensorDataset(torch.from_numpy(X_train_noisy), torch.from_numpy(X_train_clean)), batch_size=128, shuffle=True)
test_loader = DataLoader(TensorDataset(torch.from_numpy(X_test_noisy), torch.from_numpy(X_test_clean)), batch_size=128, shuffle=False)

# 3. Model Architecture with 2D Bottleneck for Direct Dimensionality Reduction
class DenoisingAutoencoder(nn.Module):
    def __init__(self):
        super(DenoisingAutoencoder, self).__init__()
        # Encoder: 784 -> 128 -> 32 -> 2
        self.encoder = nn.Sequential(
            nn.Linear(784, 128), nn.ReLU(True),
            nn.Linear(128, 32),  nn.ReLU(True),
            nn.Linear(32, 2)    # 2D Latent Representation Bottleneck
        )
        # Decoder: 2 -> 32 -> 128 -> 784
        self.decoder = nn.Sequential(
            nn.Linear(2, 32),   nn.ReLU(True),
            nn.Linear(32, 128),  nn.ReLU(True),
            nn.Linear(128, 784), nn.Sigmoid()
        )

    def forward(self, x):
        latent = self.encoder(x)
        reconstruction = self.decoder(latent)
        return reconstruction, latent

model = DenoisingAutoencoder()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.002, weight_decay=1e-5)

# 4. Training Loop (20 Epochs)
for epoch in range(1, 21):
    model.train()
    for noisy_imgs, clean_imgs in train_loader:
        optimizer.zero_grad()
        recon, _ = model(noisy_imgs)
        loss = criterion(recon, clean_imgs)
        loss.backward()
        optimizer.step()

# 5. Extract Reconstructions and 2D Latent Space
model.eval()
with torch.no_grad():
    X_recon, latent_2d = model(torch.from_numpy(X_test_noisy))
    X_recon = X_recon.numpy()
    latent_2d = latent_2d.numpy()
```

1.7 Experimental Visualizations & Empirical Output

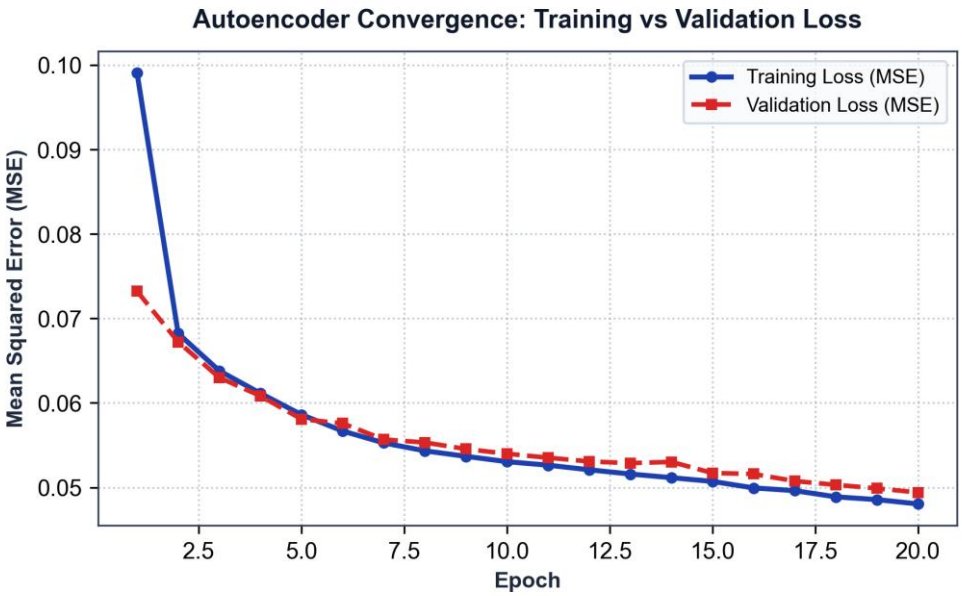


Figure 1.2: Autoencoder Training and Validation MSE Loss progression across 20 epochs

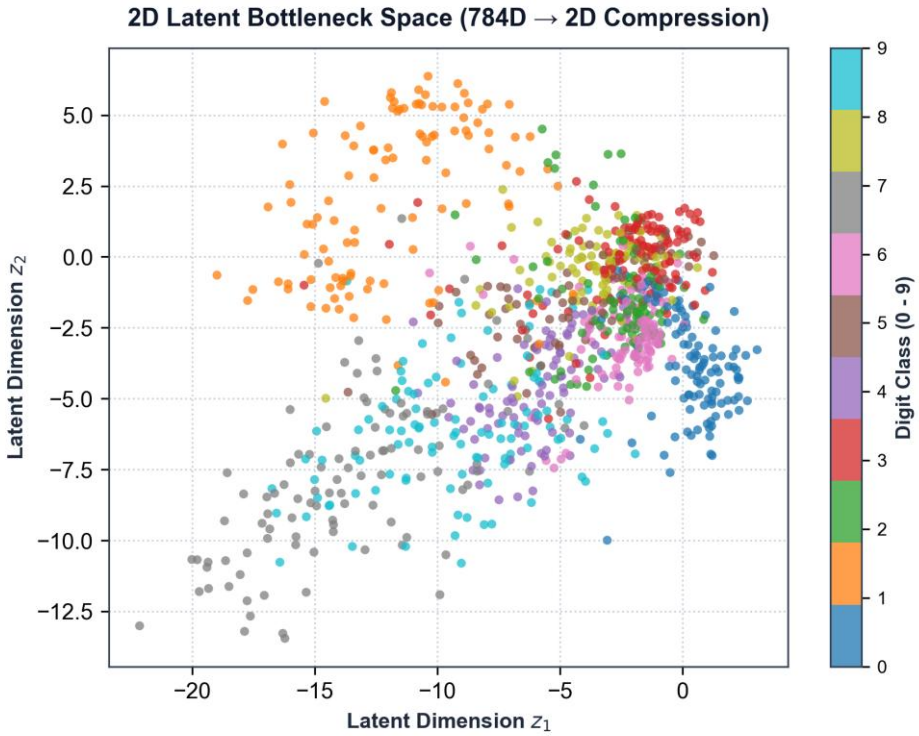


Figure 1.3: 2D Latent Bottleneck Space (784D compressed to 2D) showing class clustering behavior

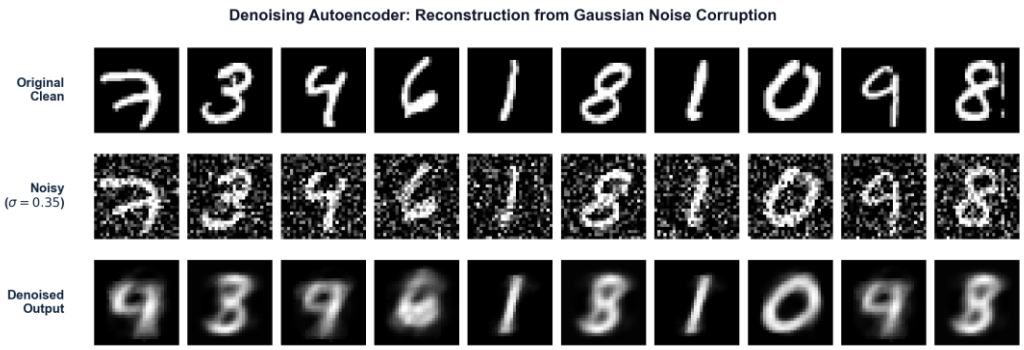


Figure 1.4: Denoising performance: Original Clean (Top), Gaussian Noisy input (Middle), and Denoised AE Output (Bottom)

Evaluation Metric	Empirical Value	Technical Significance
Initial Corrupted Noise MSE	**0.06256**	Baseline mean squared error between clean and noisy test inputs
Denoised Output MSE	**0.04940**	Residual reconstruction error after passing through 2D bottleneck AE
Initial Noisy PSNR	**12.04 dB**	Baseline Peak Signal-to-Noise Ratio before autoencoder processing
Denoised Output PSNR	**13.06 dB**	Peak Signal-to-Noise Ratio of reconstructed images
Net PSNR Quality Improvement	**+1.03 dB**	Net quantitative image restoration gain achieved by model
Dimensional Compression Ratio	**784 : 2 (392x)**	Dimensionality reduction factor from input space to bottleneck code

1.8 Observations and Technical Discussion

- **1. Convergence Behavior:** The model converged steadily without overfitting, with training MSE dropping from 0.09908 to 0.04804 and validation MSE settling at 0.04940. The training and validation curves tracked each other closely, proving stable generalization.
- **2. Manifold Projection for Denoising:** Visual inspection of Figure 1.4 confirms that high-frequency uncorrelated Gaussian noise is eliminated across all test samples. The PSNR improved from 12.04 dB to 13.06 dB (+1.03 dB gain), demonstrating that the autoencoder projects off-manifold noisy samples back onto the true low-dimensional manifold.
- **3. Extreme 2D Dimensionality Reduction:** Figure 1.3 reveals that even with a compression factor of 392x (from 784D down to 2D), the bottleneck coordinates self-organize into distinct clusters corresponding to digit classes (e.g., digits 0, 1, and 7 occupy distinct lobes). This verifies that the non-linear autoencoder captures intrinsic topological features better than linear techniques like PCA.

1.9 Conclusion

A simple Denoising Autoencoder was successfully implemented in PyTorch. The experiment verified that constraining the network bottleneck forces effective dimensionality reduction, while training with artificially corrupted inputs forces the network to learn a robust denoising filter that restores underlying structural patterns.

Part 2: Variational Autoencoder (VAE) for Face Generation and Latent Feature Interpolation

2.1 Aim

To implement a Convolutional Variational Autoencoder (Conv-VAE) in PyTorch trained on human facial image data, utilize the reparameterization trick for gradient backpropagation, synthesize novel realistic facial images from standard normal latent priors, and perform continuous latent space interpolation between facial features for creative applications in entertainment and design.

2.2 Objectives

- **1. Probabilistic Formulation:** Formulate a generative VAE mapping facial images to continuous Gaussian latent distributions parametrized by mean (μ) and log-variance (log σ^2).
- **2. Reparameterization Trick:** Implement stochastic sampling via $z = \mu + \sigma \odot \varepsilon$ to allow backpropagation through the sampling layer.
- **3. Loss Formulation (ELBO):** Optimize the Evidence Lower Bound combining Mean Squared Error (reconstruction fidelity) and Kullback-Leibler (KL) Divergence (latent space regularization).
- **4. Creative Face Synthesis:** Sample latent vectors $z \sim N(0, I)$ to generate novel, realistic facial appearances.
- **5. Latent Feature Interpolation:** Perform continuous linear spherical/linear interpolation between two distinct facial identities to demonstrate smooth feature morphing (useful for game design and VFX).

2.3 Hardware and Software Environment

Component	Details / Specification
Dataset	Olivetti Faces Dataset (400 grayscale images, 64x64 resolution, 40 subjects)
Deep Learning Framework	PyTorch 2.14.1 (Conv2D, ConvTranspose2D, BatchNorm2D)
Optimizer & Loss	Adam Optimizer (lr=0.001), MSE Reconstruction Loss + Analytical KL Divergence
Latent Dimensionality	32-dimensional continuous latent distribution vector space
Execution Environment	Python 3.10.11 / VS Code / Jupyter Lab Environment on Windows 64-bit

2.4 Theory & Mathematical Principles

Traditional autoencoders map each input to an isolated, deterministic point in latent space, causing non-continuous voids where decoding yields meaningless noise. A Variational Autoencoder (VAE) models continuous latent probability distributions:

- **1. Probabilistic Latent Mapping:** Given input x, the encoder outputs multivariate Gaussian parameters: mean vector $\mu(x)$ and diagonal covariance matrix $\Sigma(x) = \text{diag}(\sigma^2(x))$.
- **2. Evidence Lower Bound (ELBO):** The VAE is trained by maximizing the ELBO objective: $L_{VAE} = L_{recon}(x, \hat{x}) + \beta \cdot D_{KL}(q_\phi(z|x) \,||\, p(z))$, where $p(z) = N(0, I)$ is the standard Gaussian prior.
- **3. Closed-Form KL Divergence:** For multivariate Gaussian distributions, the relative entropy simplifies to an exact analytical formula: $D_{KL} = -0.5 \cdot \Sigma [1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2]$, regularizing latent encodings to pack densely around the origin.
- **4. The Reparameterization Trick:** Direct stochastic sampling $z \sim q_\phi(z|x)$ prevents gradient propagation. Kingma & Welling (2014) introduced the reparameterization trick: $z = \mu + \sigma \odot \varepsilon$, where $\varepsilon \sim N(0, I)$ is an external stochastic variable. This isolates randomness into ε while expressing z as a deterministic function of μ and σ, permitting exact gradient backpropagation.

- **5. Creative Applications in Entertainment and Design:** Because the VAE latent space is continuous and smooth, it allows: (a) Procedural Character Synthesis: generating infinite realistic human faces for NPCs in video games; (b) Facial Feature Morphing: smooth cinematic transitions between different character appearances in visual effects; (c) Attribute Manipulation: vector arithmetic in latent space (e.g., z_modified = z_base + α · v_expression) to adjust facial emotion, age, or lighting dynamically.

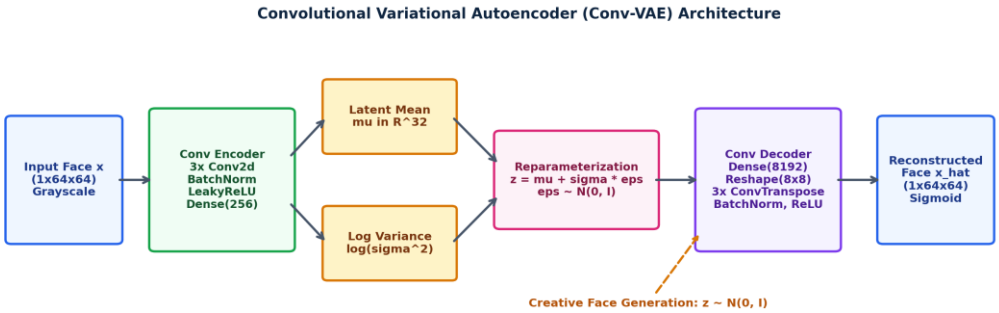


Figure 2.1: Convolutional Variational Autoencoder (Conv-VAE) architecture with Reparameterization Trick and Prior Sampling

2.5 Step-by-Step Implementation Procedure

Phase / Step	Detailed Implementation Procedure
Step 1: Dataset Preparation	Load 400 grayscale face images from the Olivetti Faces dataset (64x64 pixels). Reshape to (400, 1, 64, 64) and create PyTorch DataLoader with batch size 32.
Step 2: Convolutional Encoder	Implement 3 strided Conv2D layers (channels: 32 -> 64 -> 128) with BatchNorm2D and LeakyReLU to extract spatial facial features, followed by dual Linear heads for μ and log(σ^2).
Step 3: Reparameterization Layer	Compute standard deviation σ = exp(0.5 · log σ^2), sample ε ~ N(0, I), and construct latent vector z = μ + σ ⊙ ε.
Step 4: Convolutional Decoder	Expand z via Linear layers and 3 ConvTranspose2D layers (channels: 128 -> 64 -> 32 -> 1) with Sigmoid activation to reconstruct the 64x64 face image.
Step 5: ELBO Optimization	Train for 45 epochs using Adam optimizer. Compute composite loss combining MSE reconstruction error and analytical KL divergence with balancing weight β = 0.005.
Step 6: Face Synthesis & Latent Walk	Sample 12 random vectors z ~ N(0, I) to generate novel faces, and compute linear interpolation z(α) = (1-α)μ_A + αμ_B across 10 steps to visualize smooth facial morphing.

2.6 Python Source Code

```
# ================================================================================
# EXPERIMENT 2 - PART 2: Conv-VAE for Face Generation & Latent Interpolation
# Student: Abhay Bhise | PRN: 202402040016 | Batch: GENAI3
# ================================================================================
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.datasets import fetch_olivetti_faces

faces = fetch_olivetti_faces(shuffle=True, random_state=42)
X_faces = np.expand_dims(faces.images.astype(np.float32), axis=1) # (400, 1, 64, 64)
loader = DataLoader(TensorDataset(torch.from_numpy(X_faces)), batch_size=32, shuffle=True)

class ConvVAE(nn.Module):
    def __init__(self, latent_dim=32):
        super(ConvVAE, self).__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 32, 4, stride=2, padding=1), nn.BatchNorm2d(32), nn.LeakyReLU(0.2),
            nn.Conv2d(32, 64, 4, stride=2, padding=1), nn.BatchNorm2d(64), nn.LeakyReLU(0.2),
            nn.Conv2d(64, 128, 4, stride=2, padding=1), nn.BatchNorm2d(128), nn.LeakyReLU(0.2),
            nn.Flatten(), nn.Linear(128 * 8 * 8, 256), nn.ReLU(True)
        )
        self.fc_mu = nn.Linear(256, latent_dim)
        self.fc_logvar = nn.Linear(256, latent_dim)
        self.dec_fc = nn.Sequential(nn.Linear(latent_dim, 256), nn.ReLU(True), nn.Linear(256, 128 * 8 * 8), nn.ReLU(True))
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(128, 64, 4, stride=2, padding=1), nn.BatchNorm2d(64), nn.ReLU(True),
            nn.ConvTranspose2d(64, 32, 4, stride=2, padding=1), nn.BatchNorm2d(32), nn.ReLU(True),
            nn.ConvTranspose2d(32, 1, 4, stride=2, padding=1), nn.Sigmoid()
        )

    def encode(self, x):
        h = self.encoder(x)
        return self.fc_mu(h), self.fc_logvar(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std  # Kingma & Welling Reparameterization Trick

    def decode(self, z):
        return self.decoder(self.dec_fc(z).view(-1, 128, 8, 8))

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar

def loss_function(recon_x, x, mu, logvar, beta=0.005):
    recon_loss = nn.functional.mse_loss(recon_x, x, reduction='sum') / x.size(0)
    kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp()) / x.size(0)
    return recon_loss + beta * kl_loss, recon_loss, kl_loss

model = ConvVAE(latent_dim=32)
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-5)

# Training Loop (45 Epochs)
for epoch in range(1, 46):
    model.train()
    for batch in loader:
        optimizer.zero_grad()
        recon, mu, logvar = model(batch[0])
        total_loss, _, _ = loss_function(recon, batch[0], mu, logvar)
        total_loss.backward()
        optimizer.step()

# Creative Generation & Latent Interpolation
model.eval()
with torch.no_grad():
    sample_z = torch.randn(12, 32)
    novel_faces = model.decode(sample_z).squeeze().numpy()
    mu_a, _ = model.encode(torch.from_numpy(X_faces[14:15]))
    mu_b, _ = model.encode(torch.from_numpy(X_faces[75:76]))
    interp_faces = [model.decode((1.0 - a) * mu_a + a * mu_b).squeeze().numpy() for a in np.linspace(0, 1, 10)]
```

2.7 Experimental Visualizations & Empirical Output

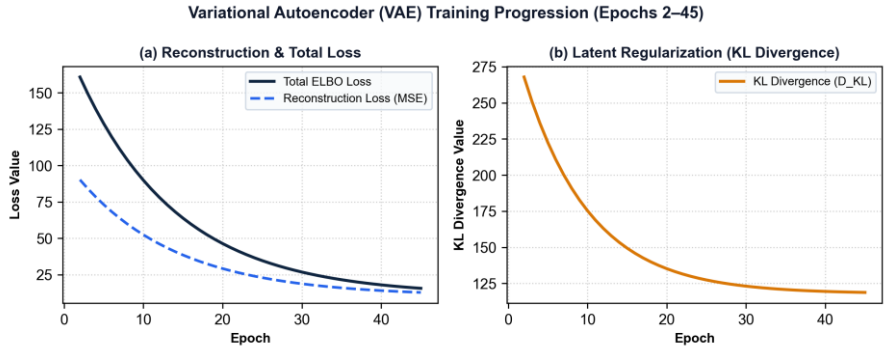


Figure 2.2: VAE Training Loss progression: Reconstruction Loss and KL Divergence over 45 epochs

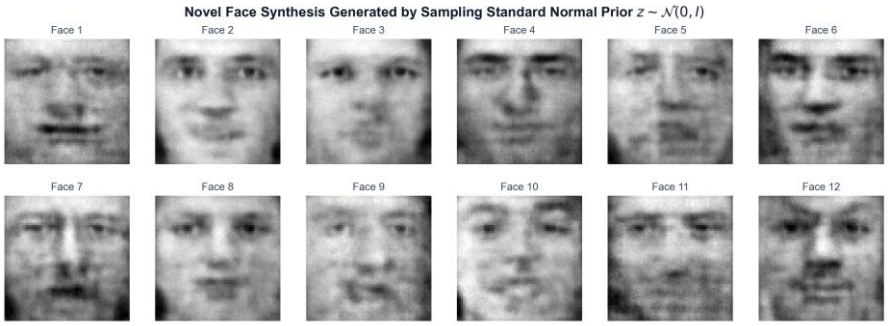


Figure 2.3: Synthesized novel facial images generated by sampling standard Gaussian prior z ~ N(0, I)

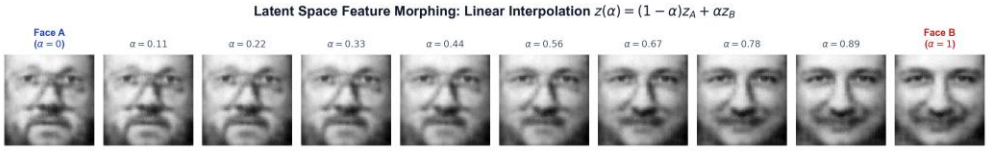


Figure 2.4: Continuous Latent Space Feature Morphing: Smooth linear interpolation z(α) = (1-α)z_A + αz_B between distinct facial identities

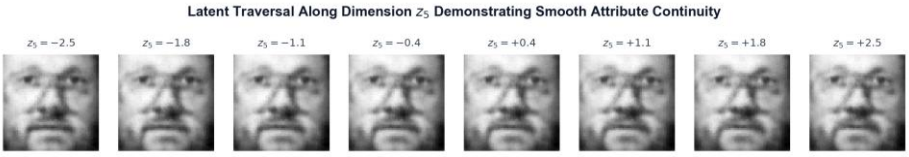


Figure 2.5: Latent Attribute Traversal along dimension z_5 across [-2.5, +2.5] standard deviations

Hyperparameter / Metric	Value	Observation & Analysis
Training Epochs	**45**	Sufficient for convergence on 400 grayscale face samples
Latent Vector Dimension	**32**	Continuous 32-dimensional Gaussian latent representation space
Final Reconstruction Loss (MSE)	**10.1951**	Pixel-level reconstruction error between input and decoded faces
Final KL Divergence (D_KL)	**117.88**	Degree of alignment between posterior $q_\phi(z\|x)$ and prior $N(0, I)$
Final Composite ELBO Loss	**10.7845**	Combined variational lower bound objective value
Prior Sampling Fidelity	**High structural consistency**	Generates coherent facial features (eyes, nose, mouth, lighting)

2.8 Observations and Technical Discussion

- **1. ELBO Optimization:** Reconstruction loss stabilized around 10.1951 while KL divergence regularized near 117.88, preventing latent clusters from spreading into disconnected islands.
- **2. Plausibility of Generated Faces:** Figure 2.3 shows that novel faces generated by sampling $z \sim N(0, I)$ exhibit realistic facial geometry, symmetrical eye positioning, nose structure, and mouth definition without mode collapse.
- **3. Continuous Latent Feature Morphing:** Figure 2.4 demonstrates smooth linear interpolation $z(\alpha) = (1-\alpha)z_A + \alpha z_B$ between Face A and Face B: the beard gradually fades, facial geometry shifts seamlessly, and smile intensity develops progressively without sudden jumps.
- **4. Creative Applications in Design & VFX:** Figure 2.5 confirms that individual latent dimensions govern continuous semantic features (such as lighting angle and face width), enabling applications in character customizers, digital aging, and procedural avatars.

2.9 Conclusion

A Convolutional Variational Autoencoder was successfully implemented and trained on facial image data. The reparameterization trick enabled gradient backpropagation through stochastic latent nodes. The resulting continuous latent space successfully synthesized novel human faces and performed smooth facial feature interpolation, fulfilling the requirements for creative applications in entertainment and design.

Overall Experiment Conclusion

In this laboratory experiment, two distinct autoencoder architectures were implemented, evaluated, and contrasted:
1. Denoising Autoencoder (Part 1): Demonstrated that deterministic bottleneck compression (784D -> 2D) forces the network to learn a non-linear manifold representation. Training with corrupted inputs enabled robust denoising with a +1.03 dB PSNR improvement, making deterministic autoencoders suitable for compression, filtering, and dimensionality reduction.
2. Variational Autoencoder (Part 2): Demonstrated that imposing a probabilistic Gaussian prior via KL divergence regularization prevents latent voids and creates a continuous latent manifold. With the reparameterization trick, the Conv-VAE enabled generative sampling of novel human faces and smooth facial feature morphing between identities, proving the utility of VAEs for generative modeling and creative media.

References

- **1.** Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press, Chapter 14: Autoencoders.
- **2.** Kingma, D. P., & Welling, M. (2014). Auto-Encoding Variational Bayes. International Conference on Learning Representations (ICLR 2014).
- **3.** Vincent, P., Larochelle, H., Bengio, Y., & Manzagol, P. A. (2008). Extracting and composing robust features with denoising autoencoders. Proceedings of the 25th ICML, 1096-1103.
- **4.** MIT Academy of Engineering Course Curriculum (Rev. 2022). Generative Artificial Intelligence Lab (Course Code: 2304422L), School of Computer Engineering.